SESSION 8: FUNCTION CALLING & TOOL INTEGRATION

Objective:

By the end of this session, students will be able to:

1. Explain why function calling is essential for LLM-based applications, including the limitations caused by knowledge cutoff and why tools are necessary to access real-time data and perform actions.

2. Describe the role and structure of tools in LLM systems, including their types (data retrieval, computation, action tools) and the key components of a tool definition (name, description, parameters schema, and output structure).

3. Demonstrate the complete lifecycle of a function call, from user query → tool selection → structured function call generation → backend execution → final LLM response.

4. Implement a single function call with Gemini API, integrating an external API (e.g., weather API) and ensuring proper schema definition for tool usage.

5. Understand and apply multiple function calling techniques, including sequential and asynchronous orchestration, and explain their relevance in real-world scenarios like travel planning or financial assistants.

6. Apply best practices for error handling and reliability, including failure isolation, timeouts, fallback strategies, schema validation (using Pydantic), and structured error communication for LLM-driven workflows.

So, till now, we've seen that LLMs are incredibly capable.
They can analyze patterns, summarize complex reports, generate ideas, even write code.
They work like a genius brain that has read almost the entire internet.

But here's the twist —
Even with all that intelligence, there's something that limits them.

The limitation is something called a **Knowledge Cutoff**.
What does this mean?
It means the model was trained on data up to a specific date, say January 2025 for Gemini.
Anything that happened after that?
The model has no idea.

'What's the current president of the United States?'
It will confidently answer… but it's just guessing, because it doesn't have access to real-time data.

Think of it like a very smart student who hasn't read today's news —
They'll try to sound correct, but most likely they'll be wrong."

So now you're thinking:
'If the model is so powerful but blind to real-time info, can we give it something extra?'
Can we take this brilliant brain and give it **hands and eyes** so it can act.

And these extra capabilities are known as Tools.

Tools are like giving the LLM superpowers:

- A weather tool → helps it see live weather data
- A calculator tool → helps solve real-time calculations
- A booking API → helps it actually make reservations

With tools, the LLM doesn't just think —
It **acts**.
It doesn't just imagine a solution —
It **executes it in the real world**."

1. Tools in LLM:

A **Tool** in the context of LLMs is an **external function, service, or API** that the LLM can call to perform a specific action or fetch real-world information.

- LLM = *brain that can think*
- Tool = *hands and eyes that can act or access live data*

The LLM sends **structured input** to the tool (usually JSON), the tool performs the action, and returns **structured output** back to the model.

These models are incredibly powerful — they can understand context, reason through logic, and even detect emotional tones.

But here's the challenge:

Let's say you want to build something real:

Use Case 1: Stock Market Analyzer

You're building a product that:

- Reads stock history

- Monitors live prices
- Detects sudden movements
- Recommends when to buy or sell

Use Case 2: Weather-Based Activity Planner

You want to build an AI that:

- Reads user preferences
- Check weather forecasts
- Picks the best activities accordingly

> "If it's raining tomorrow in Manali, suggest indoor things to do.
> If it's sunny, suggest outdoor treks."

Again, that needs:

- Live weather data
- Dynamic decision-making
- Possibly even calendar integration

Use Case 3: AI Travel Advisor

You want your GenAI system to:

- Read online reviews
- Compare hotel ratings
- Check the local climate
- Build a day-by-day itinerary

Now you're mixing:

- Real-time data (weather, ratings)
- Historical insights (user reviews)
- AI logic (planning intelligently)

So, all these use cases sound exciting, right?

But here's the **problem**:
A plain LLM, even a powerful one like Gemini or GPT-4, **cannot do any of this** on its own. Why?
Because it only knows what it was trained on.
It's like a brilliant brain locked inside a room with old newspapers.
It can imagine, reason, and predict—but it **cannot see or touch the real world**.

Now, imagine if we give this brain **superpowers**:

- Eyes to **see real-time weather**
- Hands to **pull livestock data**
- Legs to **run to your booking API and reserve your hotel**

That's exactly what **Tools** do.
They **break the LLM out of its cage** and allow it to **act on the real world**, not just think about it.

Till now, we've understood that **Tools are like the hands and eyes of an LLM** — they let a model interact with the real world instead of just predicting text.
But here's an important question:
Are all tools the same?
Of course not. Just like in real life, we use different tools for different jobs — a hammer for nails, a calculator for math, a browser for searching.
In the LLM world, tools also come in different types, each serving a specific purpose.
So, let's break it down."

1.1 Tools in LLMs: Giving AI Superpowers

Now, the next question is:
Are all tools the same?
No! Just like in real life, there are different tools for different jobs.
Let's explore them one by one in a logical flow."

1. Data Retrieval Tools → Giving AI "Eyes to See"

Transition:
"First, imagine you're building an AI travel planner. You ask:
'Plan a trip to Goa next weekend.'
The LLM gives you a nice itinerary… but the weather info? Wrong. Hotel availability? Fake.
Why? Because it's working with old data.

So the first superpower we need to give LLMs is the ability to **see the real world** through **data retrieval tools**."

Tools that fetch **real-time or external data** that the LLM doesn't have.

Real-World Scenario:

- A finance chatbot that gives you **today's stock price**.
- A news summarizer that pulls the **latest headlines**.

Without these tools, the LLM is guessing. With them, it's accurate.

Example:

- Google Search API

- Bing Search API

Okay, now we can fetch information. But what if the user says:
Convert 5,000 INR to USD and calculate compound interest for 5 years.'
Even if the LLM knows formulas, will you trust its math? Probably not.
That brings us to the next category.

2. Computation Tools → Giving AI a "Calculator Brain"

Tools that allow LLMs to **perform accurate calculations or execute code**.

Why Needed?

LLMs are great at language, but math? Not so much. They predict answers instead of calculating.

Real-World Scenario:

- Financial Advisor AI
- Budgeting Assistant
- Engineering Calculation Bot

Example Tools:

- Python REPL
- Math API
- Currency Conversion API

Now we can fetch info and calculate. But here's the catch:
What if the user says:
'Book me a hotel in Manali with 4+ star rating under ₹8,000.'
Fetching data and calculating is not enough — the AI must **act**.
And that brings us to our next category.

3. Action Tools → Giving AI "Hands to Do Things"

Definition:
Tools that let LLMs **take real actions**, like booking tickets, sending emails, or updating a calendar.

Real-World Scenario:

A personal assistant AI that:
Schedules meetings
Sends reminders
Book flights and hotels

Example Tools:

- Google Calendar API
- Gmail API
- Travel Booking API

If tools are this powerful, what do they actually look like?
Are they just some magical black box? Or is there a structure that defines them?

The answer:
A Tool is not magic—it's a well-structured definition + an implementation.

Think of it like:

- The **definition** tells the LLM *what the tool is, when to use it, and what inputs it needs*.
- The **implementation** is the actual code that runs behind the scenes to get the job done.

1.2 How does a Tool Look Like?

Think of a **Tool** like a product in an online store:

- It has a **Name** (so LLM knows what to call)
- It has a **Description** (so LLM knows when to use it)
- It has a **Specification Sheet** (the schema of input/output)

This is what we call the **Tool Card**.
This is the only part the LLM sees—it tells the model:
What is the tool for?
What are the input parameters?
What is the expected format?

1.2.1 Core Components of a Tool Definition:

When you create a tool for an LLM, you're basically giving the model a **rulebook**.
Without this rulebook, the LLM will **guess** — and when LLMs guess, things break.

Imagine you want to **hire a personal assistant**.
You don't just say:
"Go do stuff."
You give them **clear instructions**:

- What task they can perform?
- What information they need from you?
- What format they should return the answer in?

If you don't give them this clarity, they'll make assumptions and mess up.
LLMs work in the same way. As we know, **what we give to the model and how we give it** matters a lot — because these models are fundamentally **indeterministic**.

They don't truly *understand* like humans do — they **predict**. So if we leave things vague, the model starts guessing. And when it guesses, the outcome becomes unreliable or even wrong.

That's why the more **defined** and **structured** our input is, the better the output will be. And this definition we call the **Tool Definition**

Just like an API has a defined input/output format,
a tool definition tells the LLM:

- What the tool does
- What parameters (inputs) are needed
- What each parameter means and expects
- What type of output should be produced

Now to understand this better,
let's break down the core components of a tool definition using an example.
We'll build a tool called `getWeatherInfo` — a tool that fetches current weather details for a given city.

Let's see how each part of the definition works and why it matters

1.2.2 Core Components of a Tool

You just asked the model:

> "What is the weather in Gurugram today?"

The LLM can't accurately answer this because it doesn't have live data — it's limited by **knowledge cutoff**.
So, how do we fix this?
We give the LLM a **Weather Tool** that connects to a real-time weather API.

To do that, we must define the tool with these components:

1. **Name**
2. **Description**
3. **Parameters (Schema)**
4. **Output Structure**

Now, let's **build this step by step** and understand why each part matters.

1. Name – The Tool's Identity

This is like the assistant's **job title**.
It must be:

- Unique
- Clear
- Descriptive

Just like you'd call a specific app on your phone by its name (e.g., "Google Maps" or "Spotify"), our AI needs a clear name to know *which* tool it wants to use when it needs to do something. If we had a "Calculator Tool" and a "Weather Tool," the AI needs to differentiate.

Why does this matter?
If your LLM has multiple tools, names help it decide which one to call.

For Example, for our tool "`getWeatherInfo` " is a name of the tool.

Now, the tool has a name. But the LLM still doesn't know *when* to use it. If a user asks, "Should I carry an umbrella?" will the model realize whether weather is needed?

We must tell it what the tool does. That's where the **Description** comes in.

2. Description – The Tool's Job Description

Our AI, before deciding to use a tool, needs to understand *if* that tool is relevant to the user's request. The description helps the AI quickly determine if the tool's function aligns with what it needs to accomplish. It's crucial for the AI to make an informed decision about *when* to use the tool.

This description tells the LLM:
What does this tool do?
When should it be used?

Example:

"*description*": "*Fetches current or forecast weather for a given city and optional date. Use when the user asks about weather, temperature, rain chances, conditions, or whether to carry an umbrella.*

Think of this like giving context to your assistant:
"Whenever someone asks for weather details, this is your job."

Now the model knows *when* to use the tool.
But if it tries to call it without the right **details** (Which city? Which date?), the call fails.
We must define the **inputs** clearly — that's your **Parameters Schema**.

3. Parameters (Schema) – The Input Blueprint

Now, this is critical.
LLMs are probabilistic — they generate words based on patterns.
If we don't define the structure, they **guess the format**, and guesswork = broken apps.

The **schema** sets the rules:
What parameters does the tool need?
What type should each parameter be? (string, number, date)
Which ones are required vs optional?

Example for Weather Tool:

```
{
  "type": "object",
  "properties": {
    "city": {
      "type": "string",
      "description": "Name of the city (e.g., Gurugram)"
    },
    "date": {
      "type": "string",
      "description": "Date in YYYY-MM-DD format (optional)"
    }
  },
  "required": ["city"]
}
```

Now the tool can be called correctly.
But what comes *back* from the tool? Raw API JSON? Too much data? Confusing field names?
If we shape the **output**, the LLM can produce clear, consistent answers. Let's define that.

4. Output Structure – The Expected Result

Not always mandatory, but highly recommended—especially for teaching, debugging, and building reliable apps.

Why it matters:

1. **AI Comprehension:** The AI needs to understand what kind of information it's receiving back from the tool so it can process it and formulate a coherent answer for the user. If the tool just spat out a jumble of data, the AI wouldn't know what's what.
2. **Error Handling:** By defining the expected output, the AI can also detect if something went wrong (e.g., if the output isn't in the expected format, it might indicate an error with the tool or the API it called).
3. **Future Use:** Knowing the output structure allows the AI to potentially use this output as an input for *another* tool later in a more complex workflow.

Example:

```
{
  "temperature": "28°C",
  "condition": "Sunny",
  "humidity": "60%",
  "source": "WeatherAPI"
}
```

So, the LLM doesn't just return random text like:
"It's kind of sunny, I think."
Instead, you can force structured, clean data.

This is the **Tool Card** you pass to the LLM:

```
{
  "name": "get_weather_info",
  "description": "Fetches current or forecast weather for a given city and optional date. Use when the user asks about weather, temperature, rain chances, conditions, or whether to carry an umbrella.",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string",
        "description": "Name of the city (e.g., Gurugram)"
      },
      "date": {
        "type": "string",
        "description": "Date in YYYY-MM-DD format (optional; default=today)"
      }
```

```
    },
    "required": ["city"]
  }
}
```

Looks great… but let's pause.

We've created a Tool Card for our Weather Tool:

A Tool Card is *not* the tool.
It's just **information about the tool** — like a label, a menu item, or a form.
It tells the LLM *what's possible* and *what inputs are needed*, but it **does not fetch weather** by itself.

To actually *do* the work, we need the **function behind the tool** — the code that:

- Receives the structured inputs (**city**, **date**)
- Calls a weather API (or mock)
- Returns structured weather data

Until we write that function, our tool is **all brochure, no engine**.

For example:

Imagine you're a frontend developer.
You've designed a beautiful UI for a weather app — it has dropdowns for cities, a date selector, a button that says "Get Forecast," and even a placeholder for the result.

It looks polished. The layout is clean. The user can select inputs and click buttons.

But here's the catch:
You haven't written the backend yet.
No API integration. No data-fetching logic.
When the user clicks "Get Forecast," nothing actually happens.

It's all interface, no functionality.
That's exactly what a Tool Card is — just the UI schema, not the function.

Tools Work the Same Way

We need to write the actual function that:

- Accepts `city` and optional `date` as inputs

- Calls a real or mock weather API
- Returns structured weather data (like temperature, condition, etc.)

Until we write this function, the tool is **just a placeholder** — like a button with no event handler.

So yes, we've defined our **tool card** earlier:

```
{
  "name": "get_weather_info",
  "description": "Fetches current or forecast weather for a given city and optional date.",
  "parameters": {
    "type": "object",
    "properties": {
      "city": { "type": "string" },
      "date": { "type": "string" }
    },
    "required": ["city"]
  }
}
```

But **unless we create the actual function that talks to the Weather API**, this tool is useless — just like a button on the website without any event handler, it is just a UI element

That's what we'll do next.

Let's **implement the "get_weather_info" tool** in Python so the LLM can actually function, which actually performs the action.

```
import requests
from datetime import datetime


OPENWEATHER_API_KEY = "YOUR_API_KEY"


def get_weather_info(city, date=None):
    """
    Fetch current or forecast weather for a given city and optional date using OpenWeatherMap API.
```

```
    Parameters:
        city (str): Name of the city (e.g., 'Gurugram')
        date (str, optional): Date in YYYY-MM-DD format. Defaults to today.

    Returns:
        dict: Weather details including temperature, condition, and humidity
    """

    # If no date provided or it's today, fetch current weather
    if not date or date == datetime.today().strftime('%Y-%m-%d'):
        url =
f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={OPENWEATHER_API_KEY}&units=m
etric"
    else:
        # For simplicity, forecast API (next few days)
        url =
f"http://api.openweathermap.org/data/2.5/forecast?q={city}&appid={OPENWEATHER_API_KEY}&units=m
etric"

    response = requests.get(url)
    data = response.json()

    if response.status_code == 200:
        if "weather" in data:  # current weather
            return {
                "temperature": f"{data['main']['temp']} °C",
                "condition": data['weather'][0]['description'],
                "humidity": f"{data['main']['humidity']}%",
                "source": "OpenWeatherAPI"
            }
        else:  # forecast data
            return {
                "message": "Forecast available, refine logic to match date in response",
                "source": "OpenWeatherAPI"
            }
    else:
        raise Exception(f"Error fetching weather: {data.get('message', 'Unknown error')}")
```

Explanation:

1. **city (required)** → e.g., "**Gurugram**"
2. **date (optional)** → defaults to today
   - If date is today or not provided → Current Weather API
   - If future date → Forecast API
3. Returns a **structured dictionary**:

```
{
  "temperature": "28 °C",
  "condition": "clear sky",
  "humidity": "50%",
  "source": "OpenWeatherAPI"
 }
```

We've covered two big milestones so far:

1. **Tool Definition** – We created a tool card that tells the LLM:
   What is this tool? When should it be used? What inputs does it need?

2. **Tool Implementation** – We wrote the actual backend function that calls the weather API and returns structured data.

Great! So now you might be thinking:

"Cool, the tool exists. The code exists. But how does the LLM actually use this tool?"

Because here's the truth:
The LLM is **not a program runner**. It **cannot execute Python code** or **call APIs directly**.
It's a text generator. That's all.

So, how does it go from:

"What's the weather in Gurugram today?"
to actually get real data through our tool?

2. How does LLM Calls a Function?

We've defined the tool. We've implemented the code.
But the magic happens when the **LLM decides to call the tool during a conversation**.
Let's break it down.

We now know:

- **LLMs can't execute code.**
- **They rely on structured function calls to request actions.**

Let's walk through the **entire lifecycle of a user query → tool usage → final response**.

Step 1: User Initiates the Interaction

User Query:
"What's the weather forecast for Gurugram today?"

What Happens Inside the LLM?

The LLM first processes the query like a **super-intelligent language engine**:

- Detects **intent**: This question is about the *weather*.
- Identifies **information gap**: This needs *real-time data* (which the LLM doesn't have).
- Internal reasoning:
  "I can't just guess. I need to call a tool that provides weather info."

The model now needs to confirm: *Do I have a tool that can answer this kind of question?* Time to check the tool definitions.

Step 2: LLM Performs Intent & Tool Matching

The LLM now scans through the **tool definitions** provided at the start of the session.

It finds:

- **Tool Name:** get_weather_info
- **Description:**
  "Fetches current or forecast weather data for a city. Includes temperature, conditions, humidity. Optional date parameter."

Decision Process:

- Does the user request align with this tool? Yes.
- Does the query include the necessary parameters (city)? Yes: "Gurugram."
- Any optional parameter? Yes, "today" → resolve to system date.

The model has selected the right tool. Next, it must *format a request that the system can actually run*. Since it can't execute code, it produces a structured function call.

Step 3: LLM Generates a Structured Function Call

Now comes the **magic move**.
Instead of calling Python code directly (because it can't), the LLM **emits a structured instruction**—typically JSON—containing:

- **Which tool to call (name)**
- **What arguments to pass (arguments)**

Example Output:

```
{
  "name": "get_weather_info",
  "arguments": {
    "city": "Gurugram",
    "date": "2025-07-17"
  }
}
```

Why is this Step Critical?

- Natural language → **machine-readable command**.
- Reduces ambiguity → no guessing of parameter names or data types.
- Enables your backend to reliably process the request.

Now the ball is in *your* court. The LLM has requested help. Your backend must read this payload, run the actual weather function, and return data.

Step 4: Your Backend Executes the Call

Now your system steps in.

At this point, we've **received a structured response from the LLM**.
It's not plain text — it's a **JSON payload** that looks like this:

```
{
  "name": "get_weather_info",
  "arguments": {
    "city": "Gurugram",
```

```
    "date": "2025-07-17"
  }
}
```

So what does this mean?

> The LLM is saying:
> "I've understood the user's intent.
> I believe this tool — get_weather_info — is the right one to use.
> Here are the inputs I suggest."

But here's the **important point**:

> **LLMs don't execute this function — *you do*.**

What You Do in the Backend

Once you receive this function call, your backend(python code) performs 3 critical steps:

1. Identify the Function Name

You first read:

function_name = call["name"] # → "get_weather_info"

You check which function this refers to in your registered tools.

2. Extract and Validate the Arguments

Next, you pull out the arguments:

args = call["arguments"] # → {"city": "Gurugram", "date": "2025-07-17"}

Optionally, validate them:

- Is **city** non-empty?
- Is **date** in the correct format?
- Is this a supported city?

If anything is off, this is your place to raise an error or return a fallback.

3. Execute the Real Python Function

If everything looks good, you now **actually call the function**:

weather_data = get_weather_info(city="Gurugram", date="2025-07-17")

This function might:

- Call the OpenWeather API
- Handle bad input
- Parse the raw JSON from the weather service
- Return a **clean, structured output** like this:

Example Tool Output:

```
{
 "city": "Gurugram",
 "date": "2025-07-17",
 "temperature": "34°C",
 "conditions": "Sunny",
 "humidity": "55%",
 "wind_speed": "12 km/h",
 "uv_index": "High"
}
```

The **LLM never calls this function directly.**
It only makes a structured suggestion.
You — the developer — are responsible for actually running the logic.

This keeps your system secure, flexible, and controlled.

Now that your tool has done its job, you **return this result** back to the LLM.

Step 5: LLM Synthesizes the Final Answer

The **raw tool data** now flows back to the LLM as a special message.
The LLM's job:

- Convert structured JSON → **human-friendly response**.
- Preserve factual accuracy.
- Add a conversational tone if needed.

Final User Response:

"Gurugram is experiencing sunny weather today with a high of 34°C. Humidity is 55%, winds are light at 12 km/h, and UV levels are high—carry sunscreen!"

That full loop—from user language to tool call to grounded answer—is called **Function Calling**. It's how we connect LLM reasoning to real-world data and actions.

So far, we've seen how **Function Calling** works:

- The LLM analyzes a user query
- Chooses the right tool
- Creates a structured call
- Backend executes it → LLM gives a final responseGreat! But… **what if your application needs more than one tool to answer a single query?**

A **single function call** isn't enough. We need a way for the LLM to **coordinate multiple tools**, gather data from each, and combine the results into one coherent response.

We **introduce Multiple Function Calling.**

Let's level up.
We'll explore how LLMs can orchestrate **multiple tools** to solve **complex, multi-step problems**—just like a real-world travel planner does."

Note:

Orchestrate means:
"Managing and coordinating multiple tasks or actions so that they work together smoothly to achieve a final goal."

Think of a **music orchestra**.

- There are many musicians (tools/APIs).
- Each one plays a different instrument (different tasks).
- The **conductor (you or the LLM)** decides **when and how** each instrument should play.
- Together, they create a beautiful song (the final answer or output).

In AI/LLM Context:

To **orchestrate** means:

- Taking a user's request.
- Figuring out **which tools to call** (like weather API, hotel API).
- **Coordinating the calls in order (or in parallel)**.
- **Combining all the results** into one clear answer.

Example:
User asks:

"Plan a 3-day trip to Manali."

The system orchestrates by:

1. Calling **Weather API** for forecast.
2. Calling **Hotel API** for bookings.
3. Calling **Maps API** for travel routes.
4. Combining all these results into a single travel plan.

3. Multiple Function Calling – When One Tool Isn't Enough

Imagine you're planning an **international trip**.
You ask your personal assistant:
"Plan a 5-day trip to Paris next month under $2,000."

Does your assistant only need one piece of information?
No. They need to:

- **Check flights** → Which airlines, what prices, what dates?
- **Book hotels** → Available options within your budget.
- **Look at the weather** → Pack clothes accordingly.
- **Suggest activities** → Tourist spots, opening hours, entry fees.

That's **multiple tasks**, each requiring different **sources of data**. Your assistant doesn't guess—they **coordinate** all the steps, gather the info, and then combine it into **one final plan** for you.

LLMs Face the Same Challenge

In an AI application, the same thing happens.
A user might ask:
"Plan a trip from Delhi to Manali for the weekend, including weather, hotel options, and a budget estimate."

One API can't answer that.
We need:
Weather API (to check forecast)
Hotel API (to find accommodations)
Maps or Distance API (to calculate travel time and cost)

The LLM's job?
Figure out all the tools it needs, call them in the right order, and then combine their outputs into one answer.

For Example:

You're building an **AI Travel Planner**.
User asks:
"Plan a 3-day trip from Delhi to Manali next weekend with hotel options and weather updates."

Sounds simple?
Let's break down what this query actually needs:

- Weather forecast (for Manali on those dates)
- Hotel recommendations (with availability and prices)
- Travel route or transportation info
- Maybe even ratings, local attractions, and activities

That's **multiple data sources**—you can't get all of this from one API.
Weather API → Hotels API → Reviews API → Maps API.

One Function Call Won't Cut It

If we only had **get_weather_info**, the user would still be waiting for:
Hotels
Travel plans
Reviews

So, what do we do? We do multiple function calls

Multiple Function Calling is the process where an LLM:

- Detects that a single query requires **data from multiple tools**.
- Generates **multiple structured calls** to those tools, either **sequentially** (one after another) or **in parallel**.
- **Combines** the results into a single, meaningful response for the user.

3.1 How LLM Handles Multiple Function Calling

We've mastered single function calling:

- One query → One tool → One response.
  But in the real world, queries are rarely that simple.
  Let's see what happens when **a single user query needs multiple tools**.

Scenario

User asks:
"Give me today's stock price for Tesla and convert it into EUR."

This is a **multi-step problem**:

1. **Get Tesla's stock price in USD** (requires Stock Price API).
2. **Convert USD to EUR** (requires Currency Conversion API).

One API can't handle both.
So, the LLM needs to **orchestrate two tools**.

Step 1: User Initiates a Complex Query

User Query:
"What is Tesla's current stock price in EUR?"

What Happens Inside the LLM?

- Detects **intent**: Stock price query + currency conversion.
- Identifies **information gap**: Needs live stock data + exchange rate.
- Internal reasoning:
  "One tool won't cut it—I need two: Stock API and Currency API."

Step 2: LLM Matches Query to Multiple Tools

The model scans the tool definitions provided.
It finds:
get_stock_price – "Fetches real-time stock price for a given ticker."
convert_currency – "Converts amount from one currency to another."

Decision:
"Perfect. First, I'll call get_stock_price for Tesla, then pass its output into convert_currency."

Step 3: LLM Generates Multiple Structured Calls

Instead of running code, the LLM outputs **structured JSON calls**, like this:

First Call: Fetch Stock Price

```
{
  "name": "get_stock_price",
  "arguments": {
    "ticker": "TSLA",
    "currency": "USD"
  }
}
```

Second Call: Convert Currency

But it can't run this yet. It **waits for the first response**, then does:

```
{
  "name": "convert_currency",
  "arguments": {
    "amount": 252.34,
    "from_currency": "USD",
    "to_currency": "EUR"
  }
}
```

Step 4: Backend Executes Sequentially

Your system:

- Receives first JSON → Calls Stock API → Gets result (e.g., $252.34).
- Feeds that back to the LLM.
- LLM understands: "Okay, now I have price. Time for the second tool."
- Generates next function call → Calls Currency API → Gets EUR value.

Example outputs:

```
{ "price": 252.34, "currency": "USD" }

{ "converted_amount": 230.10, "currency": "EUR" }
```

Step 5: LLM Combines and Responds

Finally, the LLM merges the two results into a **single, natural response**:
"Tesla's stock is $252.34, which is approximately €230.10."

Why This Is Powerful

- **Single-function calling** handles simple queries.
- **Multiple-function calling** enables **complex orchestration**, where the LLM acts like a **project manager**, coordinating tools step-by-step.

4 Async Multiple Function Calling

Asynchronous multiple function calling means:
When multiple tools are needed, your system **starts those tool calls without waiting for each one to finish before starting the next.** You let them **run concurrently** and gather results as they complete.

You're not necessarily using multiple CPUs; you're **not blocking while waiting for slow I/O** (like network calls). That saves time.

Example

- You need: weather, hotel prices, currency rate.
- Synchronous: Call Weather API → wait. Then Hotel API → wait. Then Currency API → wait.
- Async: Fire all three at once. While Weather is loading, Hotel and Currency are also running. You wait once—for the slowest call.

4.1 When Should You Use Async Multiple Function Calling?

Use async when:

- You must call **2+ independent APIs** (weather + FX + news).
- Overall latency matters (user-facing apps, chatbots, voice).
- APIs are **I/O-bound** (network waits).
- You want to **return partial results** progressively.

Don't bother when:

- Only 1 tool needed.

- Calls are **strictly dependent** (need result A to form request B) and can't be parallelized.
- You're CPU-bound (heavy ML compute)—use multiprocessing instead.

When a user asks a complex question that needs **multiple APIs or tools** (e.g., Weather, Hotels, Currency Rates), the LLM cannot fetch these by itself. Your **backend orchestrates everything** using **asynchronous execution** to make it **fast and reliable**.

We call this the **Fan-Out / Fan-In architecture**:

- **Fan-Out** → Kick off multiple independent API calls at the same time.
- **Fan-In** → Wait for them all (or some) to complete, then combine the results.

4.2 The Full Lifecycle: Async Multi-Tool Call Flow

Step 1: User Sends a Query

Example:

"Plan a trip to Manali for next weekend. Include weather forecast, top hotels, and currency exchange rate."

At this moment:

- The user expects **one combined answer**.
- But you know **it depends on 3 different sources of truth**:
    - Weather API
    - Hotel Booking API
    - Currency Conversion API

Step 2: LLM Reasoning + Tool Selection

The LLM:

- Breaks the query into **sub-tasks**:
    - Get weather forecast → get_weather_info
    - Get hotel suggestions → get_hotels
    - Get currency rate → get_currency_rate
- Produces **structured function calls** in JSON format, one for each tool.

Example JSON emitted by the LLM:

```
[
```

```
  {
    "name": "get_weather_info",
    "arguments": { "city": "Manali", "date": "2025-07-21" }
  },
  {
    "name": "get_hotels",
    "arguments": { "city": "Manali", "check_in": "2025-07-21", "check_out": "2025-07-25" }
  },
  {
    "name": "get_currency_rate",
    "arguments": { "from_currency": "USD", "to_currency": "INR" }
  }
]
```

Step 3: Backend Orchestration (Fan-Out Phase)

Now your backend takes over:

- **Parses the tool calls**.
- Creates **async tasks** for each call:
  - Weather API
  - Hotel API
  - Currency API
- **Starts them immediately** using asyncio.create_task() or similar.

Why async?

- If each API takes 2 seconds and you call sequentially → 6 seconds total.
- Async means you **wait only once**, so the slowest call (say 2 sec) decides total time.

Step 4: Gather Results (Fan-In Phase)

- You use await asyncio.gather() to **wait for all tasks**.
- If all succeed → great, combine data.
- If some fail → still return partial data (with warnings).
- Optional:
  - Use asyncio.as_completed() to **stream partial updates to the LLM/UI**.

Step 5: Aggregate Results

- Normalize outputs into a **consistent schema**.

- Example aggregated result:

```
{

  "weather": { "temp": "18°C", "condition": "Cloudy" },

  "hotels": [ {"name": "Hotel A"}, {"name": "Hotel B"} ],

  "currency_rate": { "USD": "83.12 INR" }

 }
```

Step 6: LLM Generates Final Answer

- Send the **aggregated structured result** back to the LLM.
- LLM crafts a **human-readable, conversational response**:

"Your trip to Manali looks great! Expect cloudy weather with 18°C, top hotels include Hotel A and Hotel B, and the current USD-INR rate is 83.12."

5. Error Handling & Fallbacks for Async Multi-Tool Orchestration:

When we move from **single function calls** to **multiple async calls**, complexity skyrockets. Why? Because **multiple points of failure** enter the system. Without strong handling, your app becomes brittle, and user experience suffers.

Imagine a real-world travel assistant app where:

- Tool 1: **Weather API** → Fetch destination weather.
- Tool 2: **Hotel API** → Get hotel options.
- Tool 3: **Currency API** → Fetch live exchange rates.

User asks:

> "Plan a trip to Bali for next week. Suggest hotels, check weather, and tell me the exchange rate."

Behind the scenes:

- You trigger **3 API calls in parallel**.
- **Weather succeeds** , **Currency succeeds** , **Hotel API fails (timeout)**

If you **don't handle this properly**:

- The app **crashes** or returns **"Something went wrong"**.
- **User frustration** skyrockets because the failure was **partial**, not total.

5.1 Core Principles for Error Handling

1. Isolate Failures

What it means:
When one API call fails, the entire workflow **should not collapse**. Other successful results should still reach the user.

Why important:

- In multi-tool setups, APIs are **independent**.
- A hotel search API failing shouldn't stop weather or currency info from being shown.
- Bad design: if any API fails → entire process fails. Good design: **partial success is acceptable**.

How to implement:

- Use **try/except blocks** around each async call.
- Aggregate results with **success + error keys**.

Example Response:

```
{
 "weather": { "temperature": "28°C", "condition": "Sunny" },
 "hotels": { "error": "Hotel API timeout" },
 "currency": { "USD": "83.2 INR" }
}
```

Key Takeaway: Fail **gracefully** at the tool level, not the entire app level.

2. Use Timeouts Aggressively

What it means:
Don't let a slow API block the entire response. Set a **maximum wait time** per request (e.g., 5 seconds).

Why important:

- External APIs can be **slow or hang indefinitely**.
- Async orchestration relies on all tasks completing — a hung task breaks UX.

Best Practice:

- Define **per-request timeout** AND **overall workflow timeout**.

Key Takeaway: Never trust external APIs to respond fast. Always have a cutoff.

3. Provide Fallback Data

What it means:
When real-time data is unavailable, **don't return empty responses**. Use:

- **Cached values** (e.g., yesterday's weather)
- **Estimated values**
- **Static defaults with disclaimers**

Why important:

- Keeps the experience **predictable**.
- Builds **resilience** against failures.

Example:

```
{
 "temperature": "N/A",
 "condition": "Data unavailable",
 "source": "Fallback cache"
}
```

Fallback Patterns:
Cache → If recent data exists, use it
Alternative API → Switch provider
Approximation → "Yesterday's weather" (with disclaimer)

A user-friendly answer > ***complete failure message***.

4. Validate Schema

What it means:
Never assume the API returned **valid or complete data**. Validate it before using or sending to LLM.

Why important:

- APIs can change fields or return **corrupted data**.
- Missing or wrong types break your pipeline.

How to do it:

- Use **Pydantic models** for schema enforcement.
- Fail fast if validation fails → replace with fallback.

Example:

```
from pydantic import BaseModel, ValidationError

class WeatherData(BaseModel):
    temperature: str
    condition: str
    humidity: str

try:
    validated = WeatherData(**api_response)
except ValidationError:
    return {"error": "Invalid weather schema"}
```

Validation prevents silent data corruption.

5. Aggregate Results

What it means:
After multiple async calls finish:

- Merge **all results** (success + failures) into a single payload.
- Provide a clear structure for the LLM.

Why important:

- LLM needs a complete picture (which tools succeeded, which failed).
- Enables **graceful explanations** like:
  "Weather and currency data fetched successfully, but hotel info is unavailable."

Example Aggregated Response:

```
{
 "weather": { "temperature": "28°C", "condition": "Sunny" },
 "hotels": { "error": "Timeout" },
 "currency": { "USD": "83.2 INR" }
}
```

Key Takeaway: Users expect **a summary**, not technical errors.

6. Surface Clear Errors to LLM

What it means:
If a tool fails, **inform the LLM** in a structured way so it can:

- Explain failure in user-friendly language.
- Offer alternatives (e.g., "Want me to try again?").

Why important:

- Hiding failures = LLM makes **wrong assumptions**.
- Transparency improves UX.

Example:

```
{
 "hotels": { "error": "Hotel API timed out after 10s" }
}
```

LLM Converts This To:

> "I couldn't fetch hotel options because the service timed out. Do you want me to suggest popular hotels instead?"

Key Takeaway: Communicate failures clearly, but in a helpful tone.

Conclusions:

1. Function Calling Bridges the Gap Between Knowledge and Action

LLMs are limited by their knowledge cutoff and cannot access real-time data or execute actions independently. Function calling equips them with the ability to interact with external tools and APIs, allowing them to fetch live data, perform computations, and take actions like booking tickets or sending emails.

2. Tools Act as the LLM's "Hands and Eyes"

In the LLM ecosystem, tools extend the model's capability beyond text generation. These include:

- **Data Retrieval Tools** (e.g., weather or news APIs)
- **Computation Tools** (e.g., currency conversion, financial calculations)
- **Action Tools** (e.g., booking systems, calendar management)
  Each tool is defined by a **Tool Card** containing its name, description, input schema, and output structure.

3. Core Process of Function Calling: From Query to Action

The lifecycle involves:

- LLM detects the user's intent and identifies missing information.
- Generates a **structured function call** (JSON format).
- Backend executes the real function and returns structured data.
- LLM converts the data into a natural, user-friendly response.
  This structured approach ensures reliability, clarity, and consistency in responses.

4. Multiple Function Calling and Orchestration

Real-world queries often require more than one tool (e.g., travel planning needs weather + hotel + currency tools). LLMs can orchestrate:

- **Sequential Calls** (dependent tasks like stock price → currency conversion).
- **Asynchronous Calls** (independent tasks like weather, hotels, and exchange rates fetched in parallel).
  Async orchestration improves speed and user experience through **Fan-Out/Fan-In architecture**.

5. Error Handling and Reliability Are Critical

As complexity increases, so do failure points. Key strategies include:

- **Isolating Failures** to allow partial results.
- **Timeouts** to prevent long waits.

- **Fallback Data** for resilience.
- **Schema Validation** using libraries like Pydantic to ensure data integrity.
- **Clear Error Communication** to allow LLMs to inform users gracefully.

These guardrails make function calling robust and production-ready.